

Préparation de l'environnement de développement.

Section 1 : Installation et activation WSL2, installation distribution Debian 13, php 8.4 et Composer, paramétrages Apache.

1- Généralités sur WSL2. *Source : David Roumanet, académie de Grenoble.*

Utiliser un OS, c'est se priver de l'usage d'un autre OS.

La guerre que se livrent, Microsoft (avec Windows) et la communauté Linux (Debian, Ubuntu...) tient au fait que leurs OS respectifs sont incompatibles. Un programme Linux ne fonctionne pas sous Windows et un programme Windows nécessite un "émulateur" pour fonctionner sous Linux.

Quelques sociétés ont donc proposé des solutions, comme VMWare (vSphere), Oracle (Virtualbox) ainsi que Microsoft (Hyper-V).

Ces solutions restent lourdes, car il est nécessaire de virtualiser chaque machine Linux indépendamment : réservation de mémoire vive, coeur de processeur, disque-dur... et cela a un impact important sur les performances. Un autre inconvénient est que les VM sont isolées (disques indépendants) et ne partagent donc pas le système de fichiers : il faut passer par des protocoles réseaux ou des outils particuliers.

Microsoft cherche depuis quelques années à réduire cet écart et a conçu WSL puis WSL2 qui est fourni avec la "build" 2004 de Windows 10.

La plupart du temps, les environnements de développements sont sous Linux, tandis que les entreprises préfèrent l'administration du parc de stations sous Windows.

En proposant un noyau Linux intégré à Windows, Microsoft limite la fuite vers Linux, tout en favorisant l'administration traditionnelle (migration de poste) pour les organisations.

Le noyau fonctionne dans une machine virtuelle très légère, dont les appels systèmes sont directement reliés aux fonctions (API) Windows. Pour caricaturer, le noyau Linux pense écrire sur une partition exfs4 alors que l'API Windows réalise l'écriture sur la partition en NTFS.

Alors que WSL fonctionne correctement mais connaît quelques limitations et problèmes de performances, WSL2 améliore la latence et réduit encore les problèmes de compatibilité.

2- Installer et activer WSL2 (Windows Subsystem Linux).

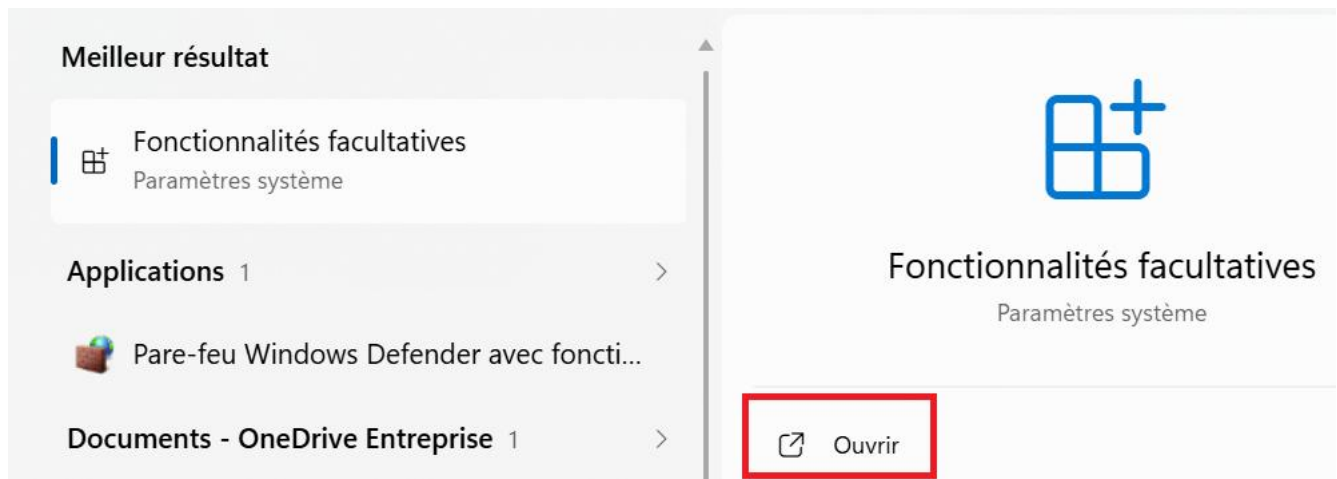
Si d'aventure vous n'êtes pas sur Windows 11, alors il faudra **une version de Windows 10 2004 adéquate**. Vérifiez que votre version de Windows est compatible.

Activation de WSL.

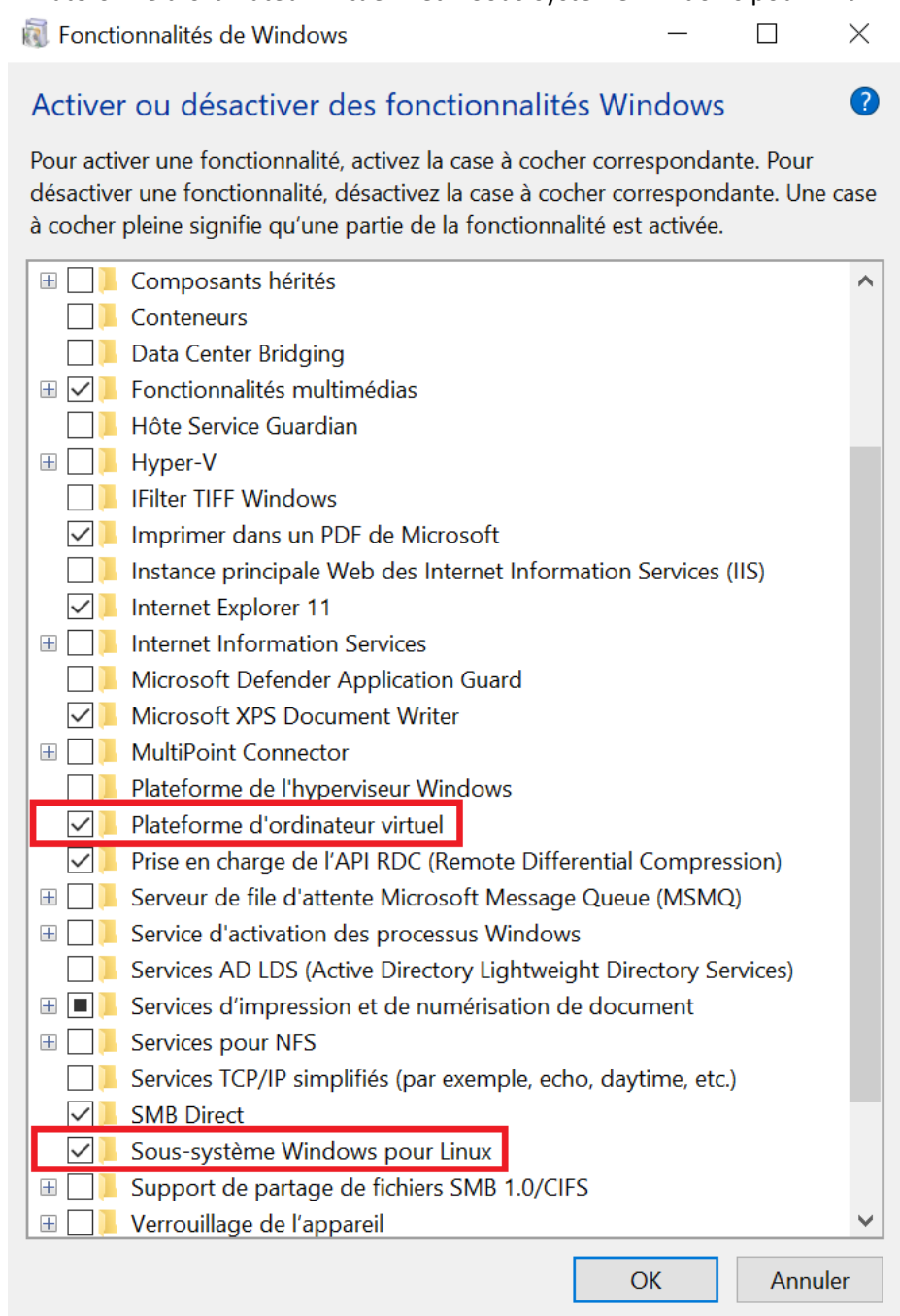
Votre machine doit redémarrer pour bénéficier de WSL.

Windows 11.

Dans la barre de recherche, taper « Fonctionnalités facultatives » :



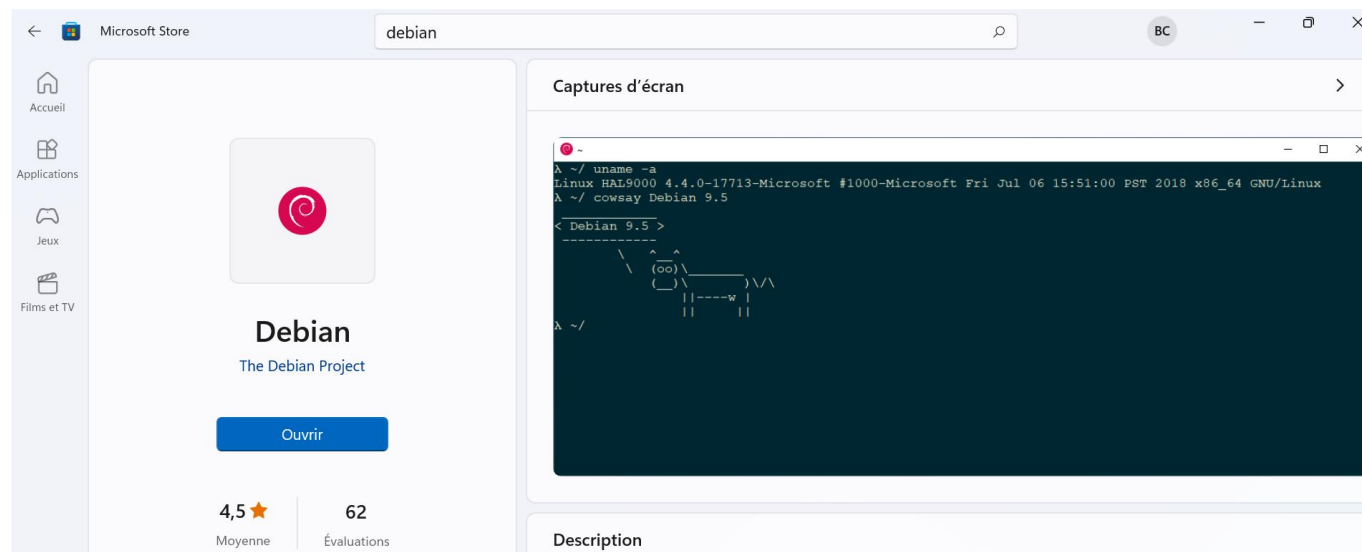
Cliquez sur « Ouvrir », scrollez jusqu'à « **Plus de fonctionnalités Windows** ». La fenêtre suivante s'ouvre, cochez « Plateforme d'ordinateur virtuel » et « Sous-système Windows pour Linux » :



Votre machine doit redémarrer pour bénéficier de WSL.

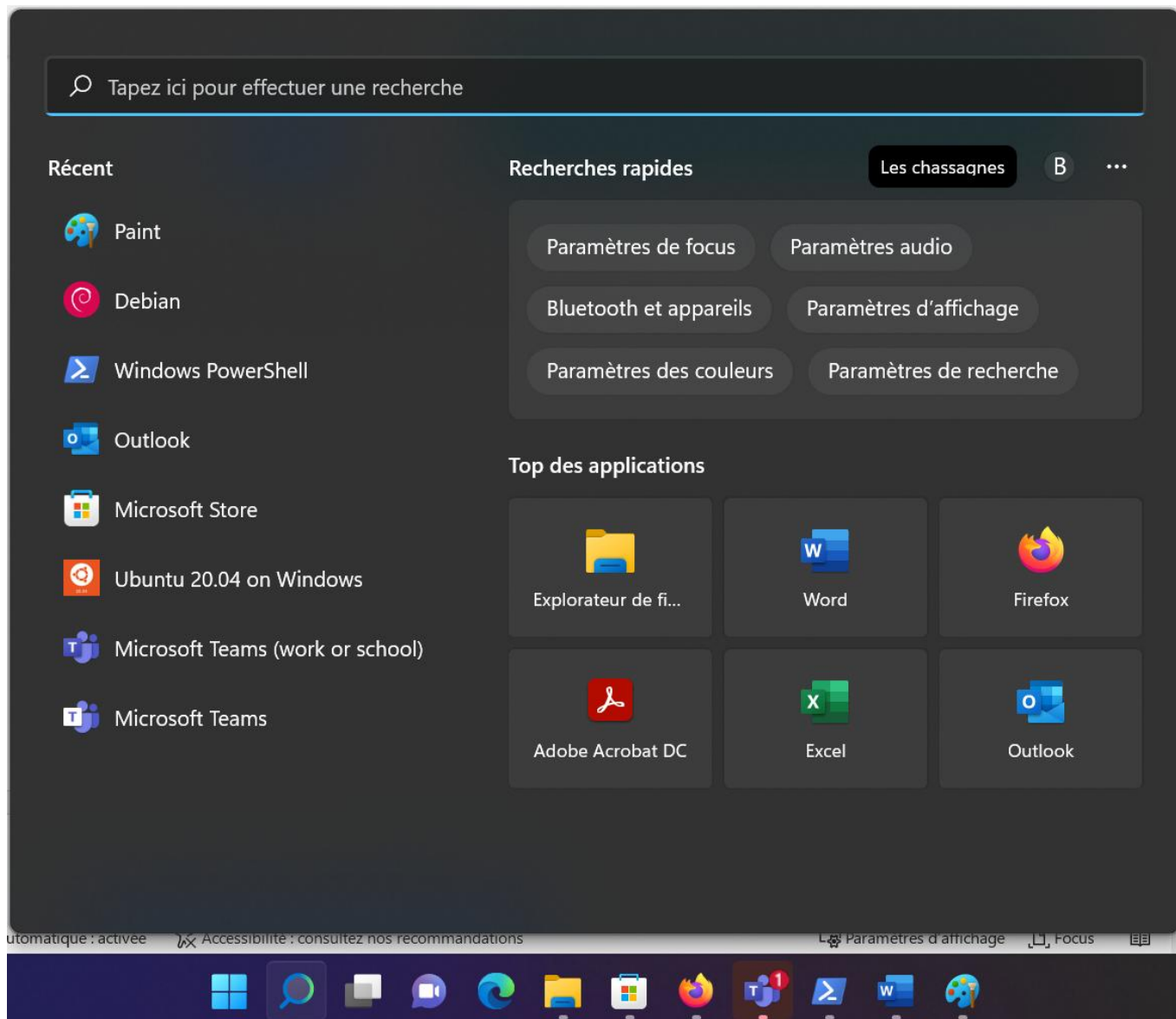
3- Installation d'une distribution Linux Debian 13 (Trixie).

Pour cette installation, rendez-vous dans le Microsoft Store et recherchez le mot-clé « Debian » :



Une fois la distribution choisie, cliquez sur le bouton « Obtenir ».

La distribution installée est accessible par le menu « Démarrer » :



Vérifiez si votre distribution Linux est définie sur wsl1 ou wsl2. Pour cela, dans Powershell, exécutez la commande : `wsl -l -v` :

```
C:\Users\bchat>wsl -l -v
  NAME                STATE      VERSION
*  Debian              Running    2
   Ubuntu-24.04        Running    2
```

4- Passage vers wsl2.

Rendez-vous à cette adresse : <https://learn.microsoft.com/fr-fr/windows/wsl/install>. Scrollez jusqu'à « Mettre à niveau la version de wsl1 vers la version wsl2. Suivez les instructions.

5- Quelques commandes utiles.

Pour sortir du terminal de debian : `exit`

Cette commande n'a pas éteint la machine qui est toujours en fonctionnement. Pour en avoir le cœur net :

Lancer Powershell et taper : wsl -l -v :



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

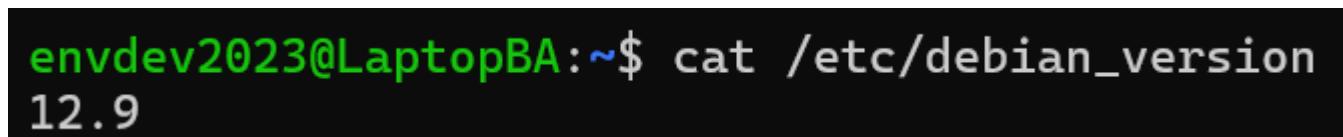
Installez la dernière version de PowerShell pour de nouvelles

PS C:\Users\bchat> wsl -l -v
  NAME                STATE      VERSION
* Ubuntu-22.04        Stopped    2
  Debian               Running    2
PS C:\Users\bchat> |
```

Pour arrêter la machine : wsl -t Debian

Pour supprimer la machine : wsl --unregister Debian puis faire la désinstallation depuis l'interface « paramètres » de Windows.

Vérification de la version Debian installée :



```
envdev2023@LaptopBA:~$ cat /etc/debian_version
12.9
```

Je ne suis pas à jour...

6- Installation de PHP 8.4, Git et la pile LAMP.

Note : 8.4 n'est pas la dernière version de PHP mais elle est encore prise en charge et très bien supportée par la dernière version du framework Laravel qui sera utilisé dans ce cours.

L'ensemble des commandes Linux à entrer pour installer la pile Lamp figure dans le fichier « installation_environnement » sur Github. Elles permettent d'installer un serveur Apache avec php et un serveur MariaDB.

Suivez à la lettre ces instructions.

7- Installation de Composer.

Composer est un logiciel gestionnaire de dépendances libre écrit en PHP. Il permet de déclarer et d'installer les bibliothèques dont le projet a besoin. (Source : wikipédia).

L'installation est décrite sur le site officiel de Composer : <https://getcomposer.org/download/>.

Pour vérifier que Composer est installé sur la machine Linux, il suffit juste de taper la commande « Composer » :

```

envdev2023@LaptopBA:~$ composer

-----
Composer version 2.6.5 2023-10-06 10:11:52

Usage:
  command [options] [arguments]

Options:
  -h, --help                Display help for the given command. When no command is given display help for the list command
  -q, --quiet               Do not output any message
  -V, --version             Display this application version
  --ansi|--no-ansi         Force (or disable --no-ansi) ANSI output
  -n, --no-interaction     Do not ask any interactive question
  --profile                Display timing and memory usage information
  --no-plugins             Whether to disable plugins.
  --no-scripts             Skips the execution of all scripts defined in composer.json file.
  -d, --working-dir=WORKING-DIR If specified, use the given directory as working directory.
  --no-cache               Prevent use of the cache
  -v|vv|vvv, --verbose    Increase the verbosity of messages: 1 for normal output, 2 for more verbose output and 3 for debug

```

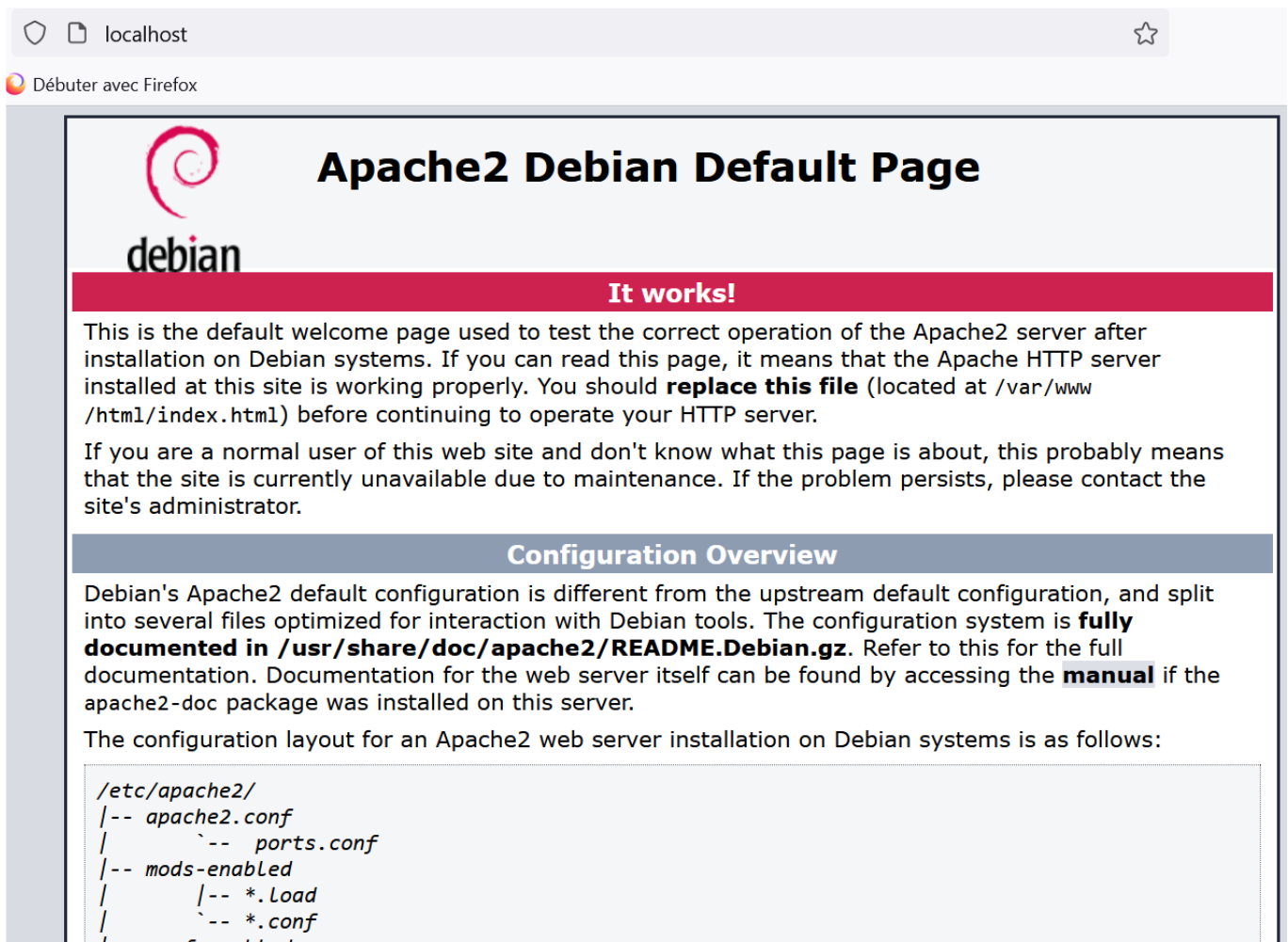
8- Les derniers réglages.

Si les deux services Apache et Mariadb ne sont pas lancés, lancez-les :

- sudo service apache2 start
- sudo service mariadb start

Vérification de l'accessibilité des deux serveurs :

Depuis Windows, on lance un navigateur et on entre l'url <http://localhost>, on obtient ceci :



Ça marche...

L'accès au serveur MariaDB peut se faire en mode console sous Linux :

```
envdev2023@LaptopBA:~$ sudo mysql
[sudo] password for envdev2023:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 34
Server version: 10.11.4-MariaDB-1~deb12u1 Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+-----+
| Database                |
+-----+
| information_schema      |
| mysql                   |
| performance_schema      |
| sys                     |
+-----+
4 rows in set (0.001 sec)

MariaDB [(none)]> |
```

Ça marche également !

Mais nous allons accéder à ce serveur via un client **windows** : **datagrip**. Vous pouvez utiliser un autre client si vous le souhaitez. Datagrip est édité par JetBrains, les étudiants de Lyon 1 peuvent l'utiliser gratuitement.

La connexion avec « root » n'est pas possible : « root » se connecte avec le plugin « unix-socket » : la connexion est limitée aux utilisateurs de Linux.

Nous allons créer un utilisateur « admin » avec tous les droits sur le serveur, mais identifié avec un mot de passe :

- 1- create user admin@localhost identified by 'P@ssW0rd' ;
- 2- grant all privileges on *.* to admin@localhost with grant option ;
- 3- flush privileges ;

```
MariaDB [(none)]> create user admin@localhost identified by 'P@ssW0rd';
Query OK, 0 rows affected (0.030 sec)

MariaDB [(none)]> grant all privileges on *.* to admin@localhost with grant option;
Query OK, 0 rows affected (0.008 sec)

MariaDB [(none)]> flush privileges;
Query OK, 0 rows affected (0.005 sec)

MariaDB [(none)]> |
```

Cet utilisateur possède les mêmes privilèges que « root » (mais il a un mot de passe !).

On peut maintenant utiliser Datagrip pour se connecter au serveur : on crée un nouveau datasource :

Data Sources and Drivers

Data Sources | **Drivers**

Project Data Sources

debian_wsl@localhost

Problems

Name: debian_wsl@localhost [Create DDL Mapping](#)

Comment:

General | Options | SSH/SSL | Schemas | Advanced

Connection type: default Driver: MariaDB [More Options](#)

Host: localhost Port: 3306

Authentication: User & Password

User: admin

Password: <hidden> Save: Forever

Database:

Succeeded [Copy](#)

DBMS: MariaDB (ver. 10.11.4-MariaDB-1~deb12u1)

Case sensitivity: plain=exact, delimited=exact

Driver: MariaDB Connector/J (ver. 3.0.7, JDBC4.2)

Ping: 32 ms

SSL: no

Le test de la connexion est OK

Test Connection ✓ MariaDB

OK Cancel Apply

On clique sur OK, on exécute une requête permettant de visualiser les bases de données sur le serveur :

Database Explorer

console console_1 x

DDL

debian_wsl@localhost 0 of 4

No schemas selected

Server Objects

Services

Database

debian_wsl@localhost

console

console_1 517 ms

console_1 517 ms

Output Result 1 x

4 rows

Database

1	information_schema
2	mysql
3	performance_schema
4	sys

9- Ultimes paramétrages sur Apache.

a- Activation de l'url rewriting :

Nous serons amenés à utiliser Laravel, un framework php. La plupart des framework php nécessite l'utilisation des règles de réécriture. Pour que cela soit possible, il faut activer le module « mod_rewrite » dans Apache. Celui-ci n'est pas activé par défaut. Pour cela, taper :

sudo a2enmod rewrite

Cette commande crée un lien rewrite.load dans le répertoire /etc/apache2/mods-enabled pointant sur le répertoire /etc/apache2/mods-available. Il faut alors redémarrer le serveur apache :

sudo service apache2 restart

b- Ajout d'un alias :

Pour avoir un environnement de développement « propre », il faut créer un répertoire qui contiendra les projets Laravel, dans /var/www puis créer un alias vers ce répertoire dans notre répertoire de travail.

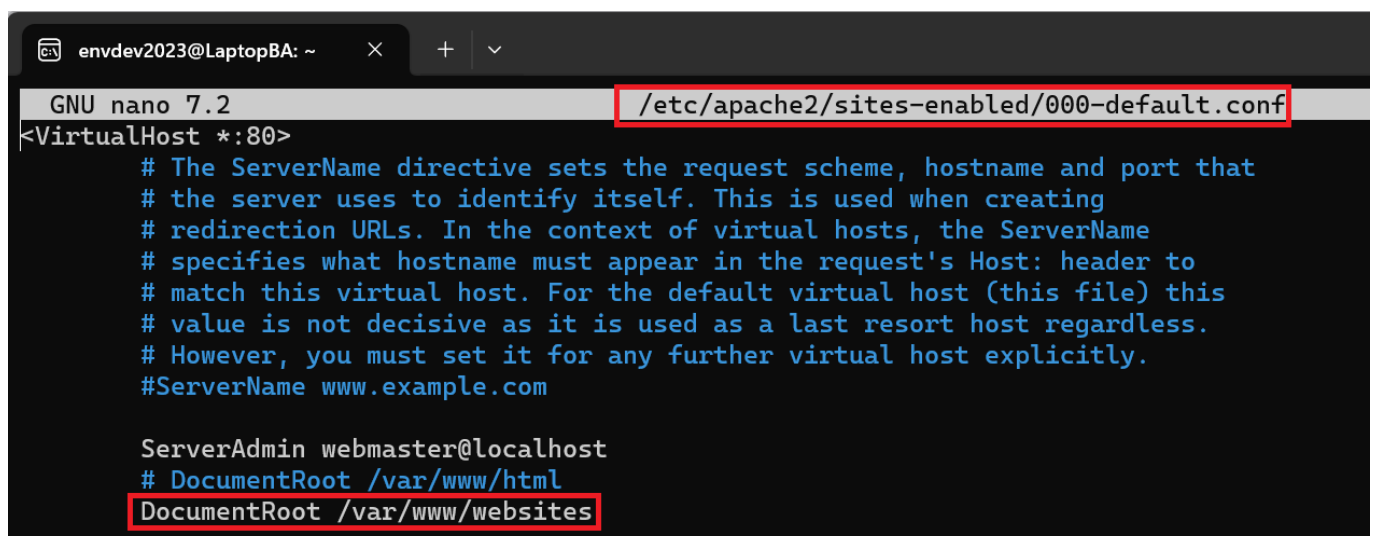
- Créer un répertoire « websites » dans /var/www.
- Se placer dans votre répertoire de travail (dans votre home).
- Taper la commande suivante : `ln -s /var/www/websites .` (*In espace moins s (sans espace) slash var slash www slash websites slash espace point*)

ATTENTION : se placer dans son répertoire de travail
(lui-même dans home) avant de taper la commande qui suit.

```
33 ln -s /var/www/websites/ .
```

Attention au point, et à l'espace entre websites et .

- ➔ On travaillera dans ce répertoire websites (qui est virtuel) situé dans notre « home », mais tout sera enregistré dans /var/www.
- ➔ Il faut alors modifier le fichier « 000-default.conf » dans /etc/apache2/sites-enabled pour mettre à jour le DocumentRoot :



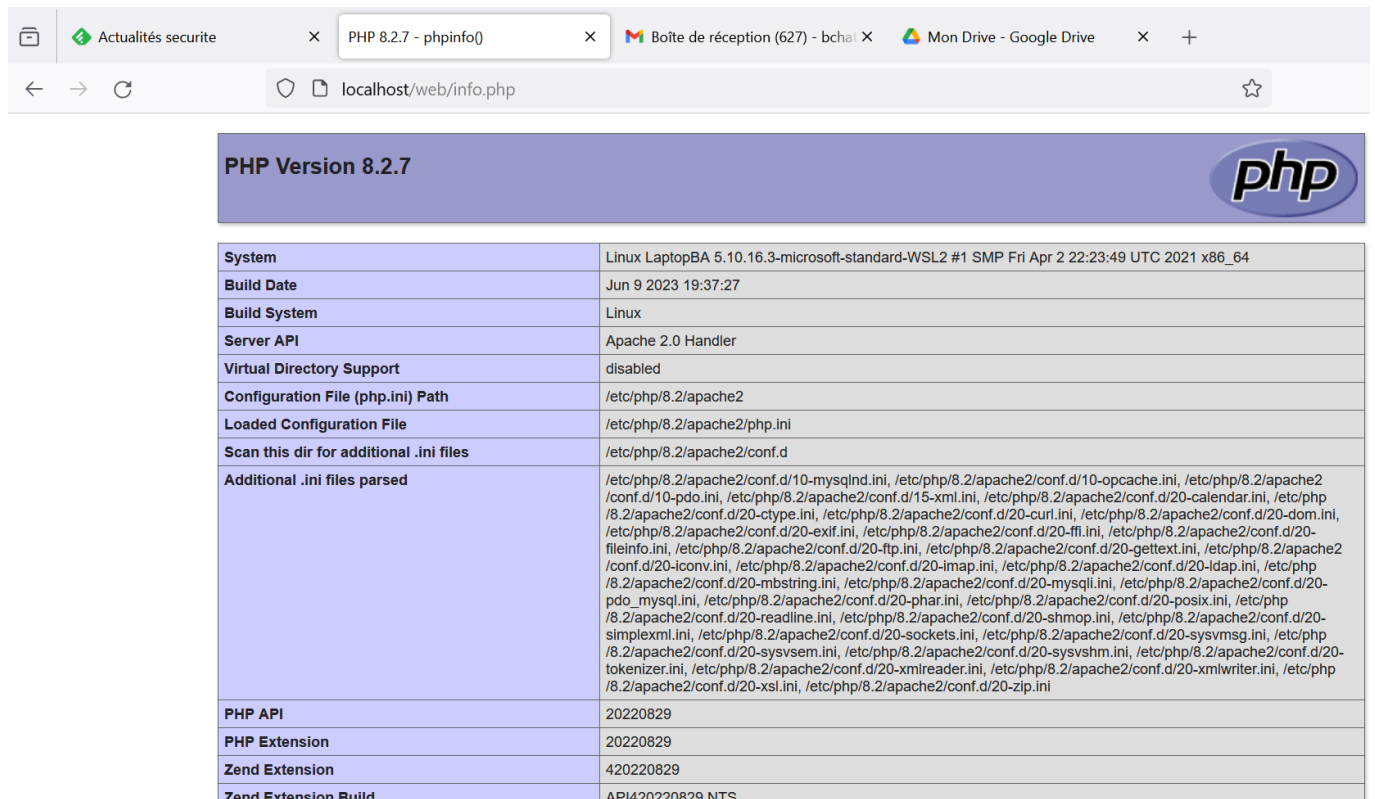
```
envdev2023@LaptopBA: ~  
GNU nano 7.2 /etc/apache2/sites-enabled/000-default.conf  
<VirtualHost *:80>  
# The ServerName directive sets the request scheme, hostname and port that  
# the server uses to identify itself. This is used when creating  
# redirection URLs. In the context of virtual hosts, the ServerName  
# specifies what hostname must appear in the request's Host: header to  
# match this virtual host. For the default virtual host (this file) this  
# value is not decisive as it is used as a last resort host regardless.  
# However, you must set it for any further virtual host explicitly.  
#ServerName www.example.com  
  
ServerAdmin webmaster@localhost  
# DocumentRoot /var/www/html  
DocumentRoot /var/www/websites
```

Redémarrer le serveur Apache.

Créer une page « info.php » avec le code suivant dans le répertoire virtuel websites :

```
< ?php phpinfo() ; ?>
```

- Tester le résultat (<http://localhost/info.php>), dans un navigateur sous Windows. On obtient ceci :



System	Linux LaptopBA 5.10.16.3-microsoft-standard-WSL2 #1 SMP Fri Apr 2 22:23:49 UTC 2021 x86_64
Build Date	Jun 9 2023 19:37:27
Build System	Linux
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/8.2/apache2
Loaded Configuration File	/etc/php/8.2/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/8.2/apache2/conf.d
Additional .ini files parsed	/etc/php/8.2/apache2/conf.d/10-mysqlnd.ini, /etc/php/8.2/apache2/conf.d/10-opcache.ini, /etc/php/8.2/apache2/conf.d/10-pdo.ini, /etc/php/8.2/apache2/conf.d/15-xml.ini, /etc/php/8.2/apache2/conf.d/20-calendar.ini, /etc/php/8.2/apache2/conf.d/20-ctype.ini, /etc/php/8.2/apache2/conf.d/20-curl.ini, /etc/php/8.2/apache2/conf.d/20-dom.ini, /etc/php/8.2/apache2/conf.d/20-exif.ini, /etc/php/8.2/apache2/conf.d/20-ffi.ini, /etc/php/8.2/apache2/conf.d/20-fileinfo.ini, /etc/php/8.2/apache2/conf.d/20-ftp.ini, /etc/php/8.2/apache2/conf.d/20-gettext.ini, /etc/php/8.2/apache2/conf.d/20-iconv.ini, /etc/php/8.2/apache2/conf.d/20-imagick.ini, /etc/php/8.2/apache2/conf.d/20-ldap.ini, /etc/php/8.2/apache2/conf.d/20-mbstring.ini, /etc/php/8.2/apache2/conf.d/20-mysqli.ini, /etc/php/8.2/apache2/conf.d/20-pdo_mysql.ini, /etc/php/8.2/apache2/conf.d/20-phar.ini, /etc/php/8.2/apache2/conf.d/20-posix.ini, /etc/php/8.2/apache2/conf.d/20-readline.ini, /etc/php/8.2/apache2/conf.d/20-shmop.ini, /etc/php/8.2/apache2/conf.d/20-simplexml.ini, /etc/php/8.2/apache2/conf.d/20-sockets.ini, /etc/php/8.2/apache2/conf.d/20-sysvmsg.ini, /etc/php/8.2/apache2/conf.d/20-sysvsem.ini, /etc/php/8.2/apache2/conf.d/20-sysvshm.ini, /etc/php/8.2/apache2/conf.d/20-tokenizer.ini, /etc/php/8.2/apache2/conf.d/20-xmlreader.ini, /etc/php/8.2/apache2/conf.d/20-xmlwriter.ini, /etc/php/8.2/apache2/conf.d/20-xsl.ini, /etc/php/8.2/apache2/conf.d/20-zip.ini
PHP API	20220829
PHP Extension	20220829
Zend Extension	420220829
Zend Extension Build	API420220829.NTS

10- Paramétrages pour débogage.

Une application php ne doit pas afficher les erreurs en mode production ; mais en mode développement, il est indispensable qu'elles le soient. Il faut modifier le fichier « php.ini » pour changer ce paramétrage.

Faire : `sudo nano /etc/php/8.2/apache2/php.ini` et rechercher la ligne `display_errors = Off`, remplacer le Off par On :

```

GNU nano 7.2 /etc/php/8.2/apache2/php.ini *
; https://php.net/error-reporting
error_reporting = E_ALL & ~E_DEPRECATED & ~E_STRICT

; This directive controls whether or not and where PHP will output errors,
; notices and warnings too. Error output is very useful during development, but
; it could be very dangerous in production environments. Depending on the code
; which is triggering the error, sensitive information could potentially leak
; out of your application such as database usernames and passwords or worse.
; For production environments, we recommend logging errors rather than
; sending them to STDOUT.
; Possible Values:
;   Off = Do not display any errors
;   stderr = Display errors to STDERR (affects only CGI/CLI binaries!)
;   On or stdout = Display errors to STDOUT
; Default Value: On
; Development Value: On
; Production Value: Off
; https://php.net/display-errors
display_errors = On

; The display of errors which occur during PHP's startup sequence are handled
; separately from display_errors. We strongly recommend you set this to 'off'
; for production servers to avoid leaking configuration details.
; Default Value: On
; Development Value: On

```

Le paramétrage d'Apache est terminé, tout doit être fonctionnel !

L'installation de l'environnement de développement est terminée.

Par la suite, tous les développements se feront dans le dossier « websites ».